

An Effective UAV Scheduling Algorithm for Public Transportation-Assisted Urban Surveillance System

Jie Zhu , Pengfei He, Ying Liu, Haiping Huang , *Member, IEEE*, Fu Xiao , *Senior Member, IEEE*,
and Reza Malekian , *Senior Member, IEEE*

Abstract—Uncrewed aerial vehicles (UAVs) are increasingly utilized in smart city applications, particularly for urban surveillance. UAVs can be provisioned as mobile surveillance to avoid various difficulties in ground operations and reduce extensive labor cost. However, their limited energy capacity restricts flight time and coverage, making it difficult to build a large-scale, long-term city-wide monitoring network. To address this problem, an ubiquitous public transportation network is introduced for UAVs to periodically recharge by landing on public transportation buses. We propose a novel public transportation-assisted UAV scheduling framework that leverages the existing bus network to enable recharging of UAVs. Two intertwined sub-problems are addressed: the UAV trajectory planning and the surveillance task offloading problems. The trajectory planning is modeled as a Traveling Salesman Problem (TSP) and it is solved via the Lin-Kernighan heuristic (LKH), decomposing the bus station network into sub-graphs for efficient routing. For task offloading, a time-slot-based scheduling method is proposed that dynamically assigns UAVs to monitor points of interest (PoIs) while ensuring energy constraints and full coverage. Experimental results demonstrate that the proposal outperforms baseline algorithms, achieving a 1.72%-3.46% average extension in system lifetime compared to state-of-the-art baselines, while maintaining computational efficiency (average runtime: 277.57 ms). The robustness of the proposal is further validated across diverse testing instances with various parameter settings.

Index Terms—Traveling salesman problem (TSP), public transportation, UAV scheduling, trajectory planning.

I. INTRODUCTION

SMART city has become the spotlight in last few decades, due to dramatic urbanization all over the world [1]. The smart city is an application of the Internet of Thing (IoT), where

Received 29 August 2024; revised 1 September 2025; accepted 5 October 2025. Date of publication 9 October 2025; date of current version 11 December 2025. This work was supported in part by the Major Research Plan of the National Natural Science Foundation of China under Grant 92467202, in part by the Jiangsu Province Universities Natural Science Research Major Project under Grant 23KJA520010, and in part by the Jiangsu Postgraduate Practical Innovation Program under Grant KYCX24_1229. Jie Zhu and Pengfei He are co-first authors. (*Corresponding author: Haiping Huang.*)

Jie Zhu, Pengfei He, and Ying Liu are with the School of Computer Science and the State Key Laboratory of Tibetan Intelligence, Nanjing University of Posts and Telecommunications, Nanjing 210013, China.

Haiping Huang and Fu Xiao are with the Jiangsu Provincial Key Laboratory of Internet of Things Intelligent Perception and Computing, Nanjing University of Posts and Telecommunications, Nanjing 210013, China (e-mail: hhp@njupt.edu.cn).

Reza Malekian is with the College of Science and Engineering, American University of Armenia, Yerevan 0019, Armenia, and also with the Department of Computer Science and Media Technology, Malmö University, 205 06 Malmö, Sweden.

Digital Object Identifier 10.1109/TSC.2025.3619675

IoT provides essential components such as data generation, data management, and application handling. In other word, smart city is an urban environment that utilizes IoT technologies to enhance performance efficiency of regular city operations and improve quality of services (QoS) provided to urban citizens. A real world smart city implementations rely upon all forms of data and computations, due to their importance in decision-making. Therefore, data collection is considered as the most important role in the smart city. The bottom layer of the smart city architecture is the sensing/data collection layer. Smart devices, wireless sensor networks (WSN), and other data capturing devices build the sensing layer.

Video surveillance system is one of important data capturing systems in the sensing layer of the smart city. Advanced video surveillance technologies allow smart city managers to obtain timely and accurate source data, keep track of real-time situations, and make flexible decisions dynamically [2]. Video surveillance data in cities is typically obtained through fixed cameras, which are not suitable for higher altitude views or quick responses to emergencies. Due to the great mobility and on-demand service ability, Uncrewed Aerial Vehicles (UAVs) have been widely used in smart cities for data collection and disaster detection [3]. Murturi et al. [4] utilize the UAVs to form an edge network for the disaster-rescue scenario. Alkous et al. [5] apply UAVs to implement the swarm-based drone delivery services. Chen et al. [6] introduce the UAVs into the edge cloud architecture as intelligent agents.

However, the limit energy capacity of UAVs severely restrict their applications. There are many kinds of charging platforms for UAVs which can be divided into two categories: the charging platforms installed at fixed locations and the charging platforms installed on mobile objects. Installing charging platforms on mobile objects such as public transportation vehicles may have different provision cost with the ones on the fixed locations. Recently, more and more buses are electrically driven and equipped with big capacity batteries which can be the power source for charging UAVs. It would be not very costly to install extra plugs (or even wireless charging platforms) on buses for charging UAVs. On the other hand, building fixed charging platforms in remote areas, where the working area of UAVs are nearby, may be a costly project. Besides, it would be more convenient to maintain the charging platforms if they are installed on buses. The maintenance personnel can maintain the charging platforms at the bus terminal when buses are idle. If the charging platforms are installed at distributed locations, it would be time consuming for the personnel to visit these charging platforms. Therefore, utilizing public transportation networks to provide charging service is an innovative practise, and it would not bring in high cost and complexity. Jin et al. [7]

introduce a new wireless charging system where both the buses and UAVs are equipped with the wireless charging platforms and provide charging service to the rechargeable sensor network. They also investigate an online electric vehicle system in [8]. The system overlays the bus network, combining the OnLine Electric Vehicle system and Microwave Power Transfer. The buses provide charging service during their regular travel to electric vehicles. Furthermore, they introduce the system into the smart health-care applications [9]. In the system, a UAV replenishes its energy by relying on the bus network. Trotta et al. [10] propose a sustainable and cost-effective recharging solution that leverages public buses to charge UAVs, ensuring the availability of city-scale video surveillance. They introduce an underlying public transportation network of a city-run fleet of buses with established routes, where buses create the energy reserves during their regular motion. By riding on buses, UAVs can replenish their energy and expand the range of surveillance services. The high penetration and wide coverage of bus networks in urban areas provide ubiquitous charging opportunities for UAVs, enabling UAV surveillance systems to sustainably provide video data support to smart cities with greater flexibility than traditional fixed-location UAV charging base stations. The above mentioned systems are expected to be commercialized in the future to enable UAV charging service. The UAV-assisted applications share the similar charging system where the buses provide charging service during their regular travel to electric vehicles or devices. The solution in these applications can reduce the cost of operation of the UAVs, however, the architecture complicates the problem of persistent coverage. In the paper, we investigate a specific transportation-assisted urban surveillance system where the buses provide charging service to UAVs and UAVs perform the surveillance service. In order to achieve the persistent coverage of the surveillance service, we consider a UAV scheduling problem where the trajectory planning and the task offloading are jointly optimized. Many points of interest (PoIs) are scattered in a given area and require either video-monitoring or video-relaying tasks, or even both. A fix number of UAVs are provisioned in the bus stations and serving for the area. These PoI tasks can be assigned to the UAVs located at the nearby bus stations. A UAV may be busy serving for a PoI at a bus station, or idle waiting for a bus to charge, or under the charging state on top of a bus. When a bus arrives at a bus station, a UAV on top of it can get off to serve for a PoI or keep charging. We do not consider the scenarios where UAVs are allowed to fly freely from one station to another one. UAVs either ride on the buses to move or hover at bus stations. The transportation-assisted urban surveillance system is continuous running if the serving states of all PoIs are not interrupted. Once a PoI is out of service, the system terminates. The duration that the system is continuous running is defined as the system life time. The objective is to maximize the system life time.

There are several challenges for the problem under study. (i) Energy management is critical for the considered problem. When a working UAV is about to out of energy, it can take a bus to charge. However, each bus has a fixed route to travel and it is possible there is no bus to take at that moment. Even there happens to have a bus to take, it should hand over its serving PoI to another available UAV. Otherwise, the PoI it serves will be out of service and the system terminates. Therefore, it is a complex computation to determine when a UAV can hand over its task and take a bus before it is out of energy. (ii) In the considered

problem, UAVs either hover at a bus station (idle or working status) or ride on buses to move (charging status). They are not allowed to fly directly from one station to another one, since the stations are generally far from each other. We cannot take the risk that a UAV may be out of power when it flies on the path to another station. Therefore, the trajectories of UAVs are difficult to plan since they are not free to go anywhere they need. UAVs have to follow the routes and timetables of the buses. When a UAV goes to the next bus station on the trajectory plan, it is possible there is no direct bus route from the current station to the target station. Multiple transfers would be necessary. Even there are multiple buses available to take, the timetable and routes of these buses should be carefully considered. (iii) It is not easy to obtain an efficient task offloading solution. Handing over a PoI to an appropriate UAV at the appropriate time is difficult. The candidate UAVs to hand over may be located at other bus stations. Both the residual energy of candidate UAVs and the time that the candidate UAVs can arrive at the bus station should be carefully considered.

The paper aims to provide an effective and efficient method for the multiple UAV scheduling problem in the transportation-assisted urban surveillance system. The main contributions of this paper are summarized as follows.

- A time-slot-based problem model integrating of trajectory planning and task offloading is formulated, where UAV trajectories are constrained by fixed-route bus networks, and task assignments must satisfy the full coverage constraint, energy constraint, and consistency constraint. The assignment (including trajectory and task offloading plans) of a UAV is modeled as a sequence of states, which are working state, idle state, and charging state. Decision variables are defined to describe these states, including the service object, the service locations, the service duration, and the start/end service time. The objective is to maximize the system lifetime as the maximal continuous coverage duration before any PoI service interruption.
- In the proposed algorithm framework, UAV trajectories are constructed first by the divide-and-conquer principle. The given original graph is not a complete digraph since not every pair of bus stations are directly connected. An artificial complete digraph is constructed by using the shortest path algorithm and then decomposed into several sub-graphs by clustering method. Each sub-graph corresponds to an independent TSP which is conquered by an effective TSP implementation. Each TSP solution is taken as a trajectory plan for one UAV.
- Based on the obtained UAV trajectories, the task offloading problem is solved. First, the initial task offloading solutions are computed using a mapping method that matches each PoI to a bus station. Second, the initial states of all UAVs are determined according to the initial task offloading solutions. Finally, the task assignments of UAVs are generated in real-time by time slot basis. In each time slot, a novel time slot scheduling method is designed to schedule UAVs and construct UAV states. The core idea is to enable all UAV resources to flow among bus stations, avoiding one UAV staying in one place for a long time.

The rest of this paper is organized as follows. In Section II, we review the related work on UAV applications and current issues. The problem is formally formulated in Section III. In Section IV, we introduce the proposed algorithm for this problem. Delicate experiments are designed to compare the

proposal and the baseline algorithms in Section V, followed by conclusions and future research in Section VI.

II. RELATED WORK

Current smart cities lack dynamic and adjustable surveillance systems, and UAVs are emerged as promising tools [11] that have been widely used in public and civil sectors to build dynamic urban surveillance networks. Compared to traditional fixed cameras, the high mobility and flexibility of UAVs offer a significant advantage, enabling surveillance from a higher and more flexible perspective [12]. However, the limited energy of UAVs severely limits the lifetime of such a system. Most of the research in this area has focused on the efficient deployment of charging stations and the replenishment of the energy of UAVs through mobile charging sites. For example, Chen et al. [13] investigate the optimal planning of charging infrastructure to expand the service range of UAV delivery. They construct a comprehensive formulation for optimal siting and sizing of UAV charging stations and propose a variable neighborhood search (VNS) with mixed integer linear programming (MILP) for the problem. Hassija et al. [14] introduce a framework for secure and reliable energy trading among UAVs and charging stations. Ma et al. [15] consider a multi-objective charging coverage problem. An assisted deep reinforcement learning model based on the survival incentive is proposed to adaptively charge UAVs while taking into account user fairness and energy-efficient deployment.

The studies mentioned above rely on charging platforms to address UAV energy limitations, which in turn increases the complexity of scheduling UAVs on these platforms. Based on this, another challenge arises: how to sustain long-term working using a UAV network. To extend the working time of UAVs, many existing studies have focused on flight path scheduling for UAV working or charging. Huang et al. [16] integrate the energy utilization, stealth and aerial mobility of UAVs, and proposes a new online trajectory planning technique for UAVs. But this approach is limited to a single UAV and does not extend to the coordinated scheduling of multiple UAVs. Rigas et al. [17] plan flight paths of UAV swarms for monitoring missions under space, time, and energy constraints, but performance deteriorates significantly when the setup is large. Wang et al. [18] investigate the problem of practical optimizing UAV trajectory in wireless charging networks. They aim to balance the total task completion time and the number of charging stations visited, so as to minimize path length and flight time, and ensure energy constraints with performance bound. Ribeiro et al. [19] define the Vehicle Route Problem (VRP) with synchronous networks and modeled the simultaneous work of UAVs and mobile charging stations to provide efficient and fast response in search and rescue missions. However, the deploying and scheduling of mobile charging stations also limits the scheduling of UAVs. Li et al. [20] study the UAV trajectory planning and task offloading problem in UAV-enabled mobile edge computing networks where UAVs are functioned as a flying energy station and edge server to provide charging and computing services for ground users. They propose a deep reinforcement learning based method to efficiently adjust task offloading policy and a energy efficiency-maximization proximal policy optimization algorithm for the optimization problem. Zhu et al. [21] investigates a UAV-aided wireless sensor networks where UAVs are dispatched to collect data from cluster heads along the planned trajectory. They propose

a novel deep reinforcement learning technique to efficiently learn from experiences from the UAV trajectory policy for minimizing the energy consumption. Wang et al. [22] design and analyze cooperative path planning algorithms of a large UAV swarm for optimally servicing many spatial locations. They transform the problem to an integer linear program by creating novel directed acyclic graph of the UAV-state transition diagram, and propose an iterative algorithm with constant approximation ratio. Jaiswal et al. [23] propose a novel approach for UAV-assisted wireless power transfer within a Radio Access Network provisioned Internet of Things (IoT) network. In their proposal, first, a K-means clustering is applied to group nodes based on their spatial proximity and energy requirements. Then a modified ACO algorithm is employed to construct efficient paths for UAVs to visit these clusters.

Employing public transportation networks to extend the range and service of UAVs is another option. Jin et al. [9] study the UAV-assisted regional disease surveillance for smart health-care where one UAV collects data from the sensors, charges the sensors to extend their life-cycles and replenishes its energy by relying on the bus network. They propose an approximation algorithm to find a feasible cycle of drone to minimize the traveling time cost of the UAV. Jin et al. [7] design a new wireless charging system, which leverages the bus network assisted drone in urban areas. An approximation algorithm DDSA is presented to find the UAV schedule with the maximal number of sensors charged by the drone before deadlines. Huang et al. [24] consider using UAVs to survey important sites across a city. UAVs can take public transportation vehicles like a passenger. Two sub-optimal solutions are designed to minimize the overall energy consumption of UAVs, subject to that every site is surveyed by a certain number of UAVs during a given time window, and all UAVs successfully return to the depot. Park et al. [25] propose a UAV task offloading scheme utilizing public buses pursuing public service objectives. An algorithm based on the successive convex approximation method is introduced to minimize the UAV system cost on energy consumption and task delay. Huang et al. [26] explore an alternative system based on a public transportation network and aim to enlarge the delivery range. A label setting algorithm and its enhanced version are designed to construct the reliable UAV path. Trotta et al. [10] design a novel urban point of interest (POI) aerial video surveillance system with the aid of a transit network. A path planning and a clustering algorithms are integrated to solve the UAV scheduling problem.

Most prior works focus on UAV trajectory planning problem across static charging stations, which introduces high infrastructure costs. Although there are some studies exploring bus-assisted UAV charging, they do not jointly optimize trajectory planning and task offloading under full-coverage constraints. Based on the aforementioned research, the paper focuses on a hybrid optimization problem consisting of UAV trajectory planning and task offloading. Bus schedules and multi-UAV coordination are considered. Unlike simple visitation tasks, surveillance systems demand that UAVs maintain area coverage at all times, which involves greater energy consumption and more stringent scheduling constraints. To address these challenges, we propose the public transportation-assisted UAV scheduling framework (PTUSF), which optimizes the lifetime of the surveillance system. Our simulation results will show that the proposed method in the paper outperforms the benchmark algorithms either on effectiveness or efficiency.

TABLE I
NOTATIONS.

Notation	Definition	Notation	Definition	Notation	Definition
$\mathbb{B}$	Set of bus stations	B_j	j^{th} bus station	M, m	Number of bus stations/buses
β	Set of buses	b_k	k^{th} bus	S_k	Setting out time of b_k
v	Driving speed of buses	r_k	Route of b_k	$r_{k,f}$	f^{th} bus station in r_k
n_k	Number of stations in r_k	$D(B_j, B_{j'})$	Distance function	j, j'	Index of bus stations
ε	One time slot	$w\varepsilon$	Bus dwell time	$T_{k,f}$	Arrival time of b_k at $r_{k,f}$
E_{max}	Maximum battery capacity	$\mathbb{P}$	Set of PoIs	H	Number of PoIs
P_h	h^{th} PoI	γ_h	Requirement of h^{th} PoI	x_h, y_h	Binary variables
$\mathcal{P}(B_j)$	Set of PoIs that can be accessed from B_j	$\mathcal{B}(P_h)$	Set of bus stations where P_h can be served	$BS(t, B_j)$	Set of buses waiting in B_j at time t
U	Set of UAVs	u_i	i^{th} UAV	n	Number of UAVs
$L_{i,0}$	Initial location of u_i	π_i	Assignment of u_i	$\pi_{i,q}$	q^{th} state of π_i
Q_i	Number of states in π_i	$p_{i,q}$	Serving object of $\pi_{i,q}$	$d_{i,q}$	Duration of $\pi_{i,q}$
$l_{i,q}, l'_{i,q}$	Start/end Location of $\pi_{i,q}$	$s_{i,q}, c_{i,q}$	Starting/Completion time of $\pi_{i,q}$	$E(\pi_{i,q})$	Energy consumption of $\pi_{i,q}$
e	Energy consumption of UAV hovering per unit time	e_m	Energy consumption of video-monitoring per unit time	e_r	Energy consumption of video-relaying per unit time
e_c	Gaining energy per unit time	$\mathbb{S}$	Schedule	$T(\mathbb{S})$	System lifetime of $\mathbb{S}$
$R(u_i, t)$	Residual energy of u_i at time t	K	Number of sub-graphs	$\mathbb{G}$	Complete digraph
$\mathbb{G}_\kappa$	κ^{th} sub-graph	$\mathbb{H}$	Circuit	$\mathbb{H}_\kappa$	Circuit of $\mathbb{G}_\kappa$
$\mathbb{M}$	Matching set of PoIs and bus stations	$\mathbb{M}_\kappa$	Matching set of PoIs and bus stations in $\mathbb{G}_\kappa$	$US(t, B)$	Set of UAVs working at bus station B at time t
C	Set of charging UAVs	W	Set of working UAVs	I	Set of idle UAVs

III. PROBLEM DESCRIPTION

Main notations in this paper are given in Table I.

In the considered problem, there exists a set $\mathbb{B} = \{B_1, \dots, B_M\}$ of M bus stations in a given area and a set $\beta = \{b_1, \dots, b_m\}$ of m buses that drive among these stations. Each bus b_k follows a specific driving route r_k with the start time S_k and the average driving speed of v ($k = 1, \dots, m$). The bus route r_k can be described by a sequence of bus stations, i.e., $r_k = (r_{k,1}, r_{k,2}, \dots, r_{k,n_k})$. Where $r_{k,f} \in \mathbb{B}$ is the f^{th} bus station in the sequence and n_k is the number of stations on the route r_k . The direct distance between two bus stations B_j and $B_{j'}$ is defined as $D(B_j, B_{j'})$, $j, j' = 1, \dots, M$. By taking the bus stations as nodes and bus routes as the paths among these nodes, the obtained graph should be a directed connected graph, i.e., there is at least one path between any two bus stations.

To model the system, we introduce a time-slotted model where $t = 0, \varepsilon, 2\varepsilon, 3\varepsilon, \dots$, and ε represents a single time slot. Each bus stops at a bus station for w time slots, which implies a bus dwell time of $w\varepsilon$. $T_{k,f}$ denotes the time that b_k arrives at the station $r_{k,f}$. $T_{k,f}$ can be defined by the recursive (1)–(2).

$$T_{k,1} = S_k \quad (1)$$

$$T_{k,f} = T_{k,f-1} + w\varepsilon + \frac{D(r_{k,f-1}, r_{k,f})}{v} \quad (2)$$

$$f = 2, \dots, n_k$$

Where $T_{k,f}$ includes the waiting time at the previous stations and the running time from the previous station to the current one.

To simply the problem model, we introduce $BS(t, B_j)$ to denote the set of buses waiting at bus station B_j at the time t . $BS(t, B_j)$ can be defined by (3).

$$BS(t, B_j) = \{b_k | b_k \in \beta, r_{k,f} = B_j, T_{k,f} \leq t \leq T_{k,f} + w\varepsilon\} \quad (3)$$

Suppose a set $\mathbb{P} = \{P_1, \dots, P_H\}$ of H PoIs are distributed around the bus stations. The problem involves two types of

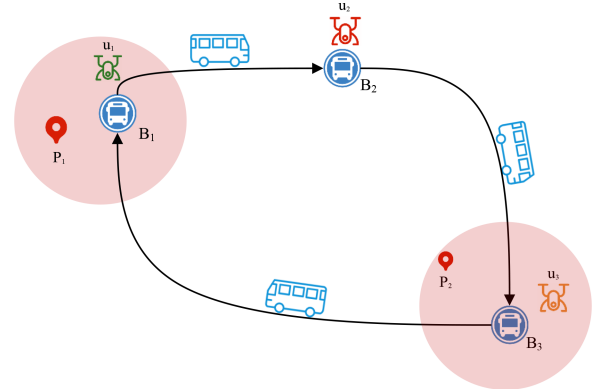


Fig. 1. An example of the considered scenario.

PoI requirements: video-monitoring and video-relaying, with some PoIs requiring both. The requirement γ_h of PoI P_h ($h = 1, \dots, H$) can be represented as a duplet $\langle x_h, y_h \rangle$, $x_h, y_h \in \{0, 1\}$. $\gamma_h = \langle 1, 0 \rangle$ means P_h only needs the video-monitoring service. $\gamma_h = \langle 0, 1 \rangle$ means P_h only needs the video-relaying service. Both services are needed if $\gamma_h = \langle 1, 1 \rangle$. The requirement $\gamma_h = \langle 0, 0 \rangle$ is not valid.

We employ $\mathcal{P}(B_j) \subseteq \mathbb{P}$ to denote the set of PoIs that can be accessed from the bus station B_j . For example, $\mathcal{P}(B_1) = \{P_1\}$, $\mathcal{P}(B_2) = \emptyset$ and $\mathcal{P}(B_3) = \{P_2\}$ in Fig. 1. It is possible that some bus station has no nearby PoI to serve, as B_2 in Fig. 1. Notation $\mathcal{B}(P_h) \subseteq \mathbb{B}$ is defined as the set of bus stations where PoI P_h can be served. In Fig. 1, $\mathcal{B}(P_1) = \{B_1\}$ and $\mathcal{B}(P_2) = \{B_3\}$. We assume every PoI should be accessed from at least one bus station, i.e., the full coverage of service for PoIs must be guaranteed.

UAVs $u_1, u_2, \dots, u_n$ equipped with GPS and WiFi modules are available for the video-monitoring and video-relaying services. Each UAV is fully charged initially and the maximum battery capacity is E_{max} . The initial location of u_i ($i = 1, \dots, n$) is denoted as $L_{i,0}$ and it must be some bus station, i.e., $L_{i,0} \in \mathbb{B}$.

Noted that the locations of UAVs are variables to be determined. We introduce $US(t, B_j)$ to denote the set of UAVs located at the bus station B_j at time t .

One UAV u_i can be in one of the following states at any given time slot:

- **Working state.** u_i is under the working state if it is currently serving a PoI at a bus station.
- **Idle state.** u_i is under the idle state if it is waiting idly at a bus station.
- **Charging state.** u_i is under the charging state if it is charging on top of a bus.

A UAV may stay under one state for a period of time and change to another state once its serving object or the location changes. The q^{th} state of u_i can be denoted as $\pi_{i,q} = (p_{i,q}, l_{i,q}, l'_{i,q}, s_{i,q}, d_{i,q}, c_{i,q})$, which means u_i starts to serve $p_{i,q}$ at time $s_{i,q}$ from the location $l_{i,q}$. The state ends at time $c_{i,q}$ with the location of u_i being $l'_{i,q}$ at that time. The duration of the state is $d_{i,q}$. $\pi_{i,q}$ is the working state if $p_{i,q} \in \mathbb{P}$. In the working state, $l_{i,q} = l'_{i,q}$ and $l_{i,q}, l'_{i,q} \in \mathbb{B}$. We use P_0 to denote a dummy PoI. If $p_{i,q} = P_0$, it means the UAV is idle at the bus station. In the idle state, $l_{i,q} = l'_{i,q}$ and $l_{i,q}, l'_{i,q} \in \mathbb{B}$. If $p_{i,q} \in \beta$, u_i is under the charging state on top of $p_{i,q}$. $p_{i,q}$ is a bus stopping at the bus station $l_{i,q}$. A UAV can land on a bus only when the bus stops at the bus station. We do not consider the scenario that a UAV can land on a moving bus. At the end of the charging state, both the bus and the UAV arrives at the bus station $l'_{i,q}$. The assignment of UAV u_i can be represented as a sequence of states, i.e., $\pi_i = (\pi_{i,1}, \pi_{i,2}, \dots, \pi_{i,Q_i})$, where Q_i represents the total number of states u_i goes through during the system lifetime.

If the state $\pi_{i,q}$ is known, the energy consumption during the state's duration can be computed by (4).

$$E(\pi_{i,q}) = \begin{cases} (e + e_m x_h + e_r y_h)(c_{i,q} - s_{i,q}) & p_{i,q} = P_h \in \mathbb{P} \\ e\varepsilon & p_{i,q} = P_0 \\ -e_c \varepsilon & p_{i,q} \in \beta \end{cases} \quad (4)$$

In (4), e represents the energy consumed by a UAV hovering per unit time, while e_m and e_r are the energy consumed by the video-monitoring and video-relaying services per unit time, respectively. e_c denotes the energy gained per unit time when a UAV is recharging on top of a bus. The energy consumption under the charging state is the negative value of the gaining energy. Generally, e_c is much less than e , e_m and e_r . It means a UAV takes longer to charge then to serve.

The schedule $\mathbb{S}$ is the set of assignments of all UAVs, i.e., $\mathbb{S} = \{\pi_1, \dots, \pi_n\}$. The schedule $\mathbb{S}$ is feasible during time $[0, T(\mathbb{S})]$ if and only if the following constraints are satisfied. $T(\mathbb{S})$ is the system lifetime which is defined as the maximum time slot that the schedule $\mathbb{S}$ is feasible on all constraints.

- **Full coverage constraint:** Every PoI should be served at any time by only one UAV. The constraint can be defined by (5)–(6). (5) indicates that the union of all time intervals when P_h is served should cover the entire system lifetime duration. (6) ensures that the time intervals when the same PoI is served do not overlap, meaning that only one UAV can serve for a PoI at any given time.
- **Energy constraint:** The residual energy of a UAV cannot be negative at any time. The constraint can be defined by (7)–(10). The residual energy of u_i at time t is defined as $R(u_i, t)$. (7) and (8) restrict that the residual energy of u_i cannot be negative at the beginning and the completion time

of any state. Initially, the UAV is fully charged (9) and its residual energy is updated in each stage (10). Apparently, the residual energy of a UAV increases during the charging state, but it cannot exceed the maximum limit $E_{\max}$.

- **Consistency constraint:** The locations and timetables of UAV states should be consistent with these of buses. Given a charging state $\pi_{i,q}$, (11)–(12) guarantee the bus $p_{i,q}$ is available for charging during the entire state. Besides, if u_i is serving for an PoI in a working state $\pi_{i,q}$, the PoI must be accessible from the bus station $l_{i,q}$ (13).

$$\bigcup_{\forall \pi_{i,q}, p_{i,q} = P_h} [s_{i,q}, c_{i,q}] = [0, T(\mathbb{S})] \quad (5)$$

$$h = 1, \dots, H$$

$$[s_{i,q}, c_{i,q}] \cap [s_{i',q}, c_{i',q}] = \emptyset \quad (6)$$

$$\forall p_{i,q} = p_{i',q'}, p_{i,q} \in \mathbb{P}, p_{i',q'} \in \mathbb{P}$$

$$\forall \pi_i, \pi_{i'}$$

$$R(u_i, s_{i,q}) \geq 0 \quad (7)$$

$$R(u_i, c_{i,q}) \geq 0 \quad (8)$$

$$R(u_i, 0) = E_{\max} \quad (9)$$

$$R(u_i, c_{i,q}) = \min\{R(u_i, s_{i,q}) - E(\pi_{i,q}), E_{\max}\} \quad (10)$$

$$p_{i,q} \in BS(s_{i,q}, l_{i,q}) \quad (11)$$

$$p_{i,q} \in BS(c_{i,q}, l'_{i,q}) \quad (12)$$

$$\forall \pi_{i,q}, p_{i,q} \in \beta$$

$$p_{i,q} \in \mathcal{P}(l_{i,q}), \forall \pi_{i,q} \in \mathbb{S}, p_{i,q} \in \mathbb{P} \quad (13)$$

The objective of the considered problem is to obtain an optimal schedule with the maximum system lifetime. The system lifetime $T(\mathbb{S})$ is defined by (15). The problem model can be formulated as following.

$$\text{Maximize } T(\mathbb{S}) \quad (14)$$

$$T(\mathbb{S}) = \max\{c_{i,Q_i} | i = 1, \dots, n\} \quad (15)$$

$$\mathbb{S} = \{\pi_1, \dots, \pi_n\} \quad (16)$$

$$\pi_i = (\pi_{i,1}, \dots, \pi_{i,Q_i}) \quad (17)$$

$$\pi_{i,q} = (p_{i,q}, l_{i,q}, l'_{i,q}, s_{i,q}, d_{i,q}, c_{i,q}) \quad (18)$$

$$c_{i,q} = s_{i,q} + d_{i,q} \quad (19)$$

$$s_{i,q} = c_{i,q-1} \quad (20)$$

$$s_{i,1} = 0, l_{i,1} = L_{i,0} \quad (21)$$

$$\text{s.t. Eqs (5) – (13)} \quad (22)$$

Fig. 1 depicts an example of the considered problem scenarios. The scenario includes three bus stations (B_1 , B_2 , and B_3) and two PoIs (P_1 and P_2), with P_1 accessible from B_1 and P_2 accessible from B_3 . One bus follows the circular route $B_1, B_2, B_3, B_1, B_2, B_3, \dots$. The timetable of the bus is given in Table II. Three UAVs are provisioned in these bus stations as shown in the figure. Two feasible schedules $\mathbb{S}_1$ and $\mathbb{S}_2$ are detailed respectively in Tables III and IV. The UAVs follow the trajectories shown in Fig. 2. The corresponding Gantt charts

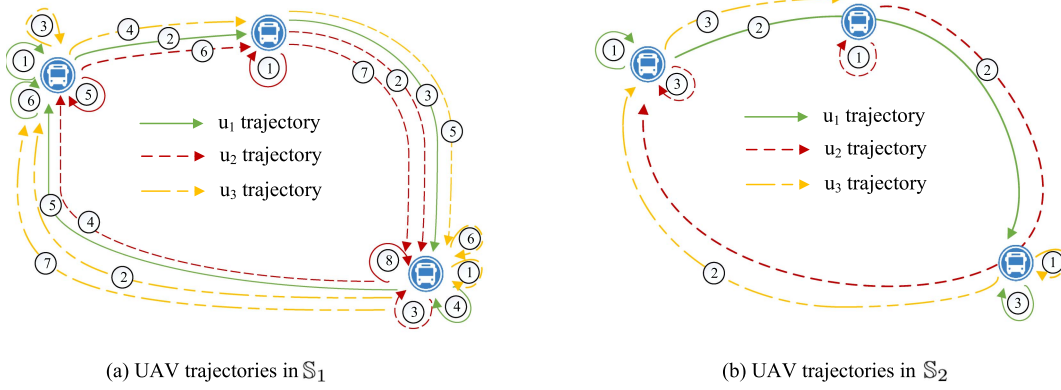


Fig. 2. UAV Trajectories.

TABLE II
BUS TIMETABLE.

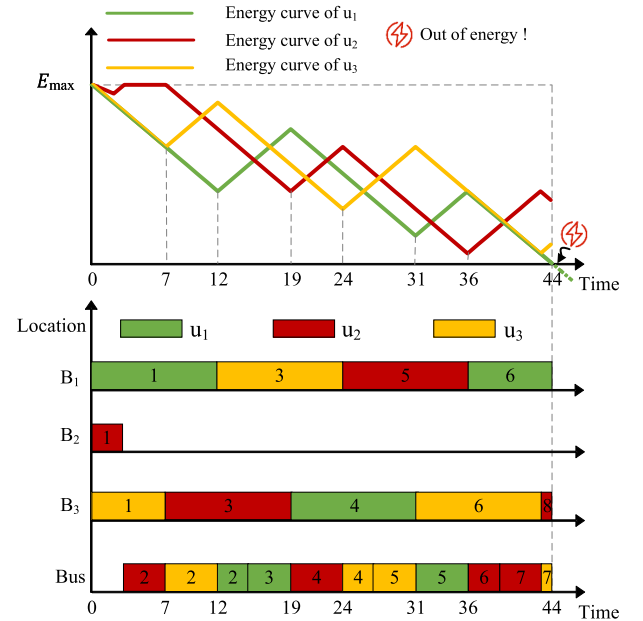
Location	Arrival Time	Departure Time
B_1	0	1
B_2	3	4
B_3	7	8
B_1	12	13
B_2	15	16
B_3	19	20
B_1	24	25
B_2	27	28
B_3	31	32
...	...	...

TABLE III
SCHEDULE $\mathbb{S}_1$.

UAV	$\pi_{i,q}$	$p_{i,q}$	$l_{i,q}$	$l'_{i,q}$	$s_{i,q}$	$d_{i,q}$	$c_{i,q}$	State
u_1	$\pi_{1,1}$	P_1	B_1	B_1	0	12	12	Working
	$\pi_{1,2}$	b_1	B_1	B_2	12	3	15	Charging
	$\pi_{1,3}$	b_1	B_2	B_3	15	4	19	Charging
	$\pi_{1,4}$	P_2	B_3	B_3	19	12	31	Working
	$\pi_{1,5}$	b_1	B_3	B_1	31	5	36	Charging
	$\pi_{1,6}$	P_1	B_1	B_1	36	8	44	Working
u_2	$\pi_{2,1}$	P_0	B_2	B_2	0	3	3	Idle
	$\pi_{2,2}$	b_1	B_2	B_3	3	4	7	Charging
	$\pi_{2,3}$	P_2	B_3	B_3	7	12	19	Working
	$\pi_{2,4}$	b_1	B_3	B_1	19	5	24	Charging
	$\pi_{2,5}$	P_1	B_1	B_1	24	12	36	Working
	$\pi_{2,6}$	b_1	B_1	B_2	36	3	39	Charging
	$\pi_{2,7}$	b_1	B_2	B_3	39	4	43	Charging
	$\pi_{2,8}$	P_2	B_3	B_3	43	1	44	Working
u_3	$\pi_{3,1}$	P_2	B_3	B_3	0	7	7	Working
	$\pi_{3,2}$	b_1	B_3	B_1	7	5	12	Charging
	$\pi_{3,3}$	P_1	B_1	B_1	12	12	24	Working
	$\pi_{3,4}$	b_1	B_1	B_2	24	3	27	Charging
	$\pi_{3,5}$	b_1	B_2	B_3	27	4	31	Charging
	$\pi_{3,6}$	P_2	B_3	B_3	31	12	43	Working
	$\pi_{3,7}$	b_1	B_3	B_1	43	1	44	Working

TABLE IV
SCHEDULE $\mathbb{S}_2$.

UAV	$\pi_{i,q}$	$p_{i,q}$	$l_{i,q}$	$l'_{i,q}$	$s_{i,q}$	$d_{i,q}$	$c_{i,q}$	State
u_1	$\pi_{1,1}$	P_1	B_1	B_1	0	12	12	Working
	$\pi_{1,2}$	b_1	B_1	B_3	12	7	19	Charging
	$\pi_{1,4}$	P_2	B_3	B_3	19	13	32	Working
u_2	$\pi_{2,1}$	P_0	B_2	B_2	0	3	3	Idle
	$\pi_{2,2}$	b_1	B_2	B_1	3	9	12	Charging
	$\pi_{2,4}$	P_1	B_1	B_1	12	20	32	Working
u_3	$\pi_{3,1}$	P_2	B_3	B_3	0	19	19	Working
	$\pi_{3,2}$	b_1	B_3	B_2	19	8	27	Charging
	$\pi_{3,3}$	b_1	B_2	B_1	27	5	32	Charging

Fig. 3. Gantt charts and energy curves of $\mathbb{S}_1$.

and energy curves of the schedules are given in Figs. 3–4. As the energy curve of u_3 shown in Fig. 3, u_3 is about to out of energy at time slot 44 and no bus is available for u_3 to recharge at the moment. Similarly in Fig. 4, u_2 is out of energy at time

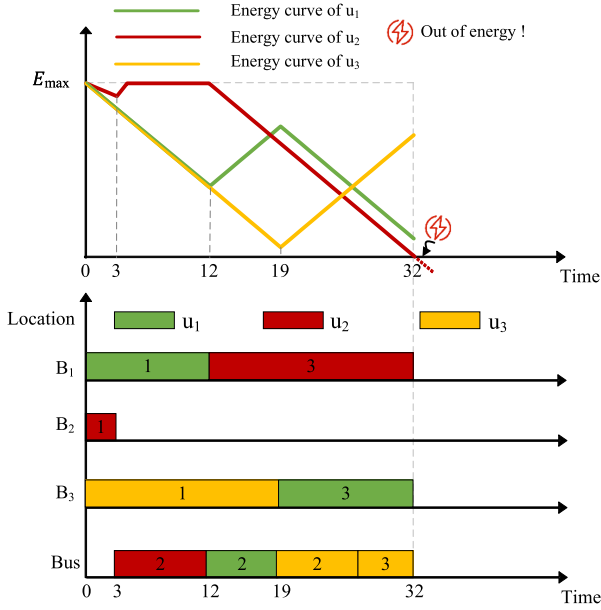


Fig. 4. Gantt charts and energy curves of S_2 .

slot 32 in S_2 . Therefore, the system lifetimes of S_1 and S_2 are 44 and 32, respectively. It is evident that S_1 outperforms S_2 in terms of system lifetime.

According to the above example, we can find out that the schedule for a UAV is not only a sequence of locations to visit, but also a detailed timetable and travel routes for each visit. In the problem considered, UAVs cannot fly by themselves. Buses act as shared mobility platforms for UAVs. The UAV must navigate a fixed-route, scheduled bus network to reach its destination, possibly requiring transfers. Unlike conventional UAV path planning, where aerial routes can be freely optimized, the UAVs in the considered problem are constrained by the fixed-route bus network and must adhere to predefined bus timetables. Therefore, there are so many decision variables that define all states for all UAVs. Tables III and IV contain all the decision variables to form a schedule for the example problem. These examples show the necessity of carefully designed scheduling algorithms.

IV. PROPOSED METHOD

We consider a UAV scheduling problem in the transportation-assisted urban surveillance system. The system involves many aspects, such as controlling, communicating, UAV scheduling, monitoring, etc., which make the system design complicated. We focus on discussing the scheduling problem in the system. In the paper, a Public Transportation-assisted UAV Scheduling Framework is proposed for the problem under study, where an effective implementation for solving the traveling salesman problem (TSP) is integrated into the framework for UAV trajectory planning. The proposed algorithm generates the scheduling solution based on the bus timetable and the states of UAVs. It can be used to generate the scheduling solution in advance. It can also be employed to generate the real-time UAV scheduling solution by updating the bus timetable and states of UAVs in real-time if some uncertain events take place to make the initial solution unfeasible. In the following subsections, we first introduce the

major framework of the proposed method. Then the details for the key components of the framework are given.

A. Algorithm Framework

Algorithm 1 describes the framework of the proposed algorithm. The major steps of PTUSF are as follows:

- 1) A complete digraph is constructed by the Complete Digraph Construction Method (CDCM) (Line 1).
- 2) The obtained complete digraph is decomposed into K sub-graphs by Graph Decomposition Method (GDM), where the scheduling problem in each sub-graph is conquered separately.
- 3) For each sub-graph, a TSP is modeled based on the obtained graph. We randomly take a bus station as an origin and call the TSP-based Heuristics (TSPH) to generate a TSP solution, which will be further taken as a trajectory plan for an idle UAV to move (Line 4).
- 4) PoIs and bus stations in each sub-graph are matched by PoI-Station Mapping Method (PISMM) (Line 5). Each PoI is initially served by a UAV located at the matched bus station.
- 5) The initial states of all UAVs are determined (Line 6).
- 6) UAVs are classified into three sets according to their initial states (Lines 7-16).
- 7) UAVs are scheduled in real-time on a time slot by time slot basis until the system is terminated (Lines 17-25). For each time slot, the states of UAVs are updated and the feasibilities of UAVs are checked. If a UAV is under an unfeasible state, i.e., the energy constraint is violated, then the system is terminated. Otherwise, the UAVs provisioned in each sub-graph are scheduled by the Time Slot Scheduling Method (TSSM). TSSM schedules all UAVs for the current time slot according to their real-time states. TSSM is called on a time slot by time slot basis and it computes scheduling solutions based on the real-time UAV states. Therefore, the obtained scheduling solution by TSSM must be feasible even if any uncertain event takes place, e.g., bus delay.
- 8) The system life time is computed and returned (Lines 26-27).

As can be seen, there are six key components in the main framework: (1) Complete Digraph Construction Method (CDCM) to build a complete digraph, (2) Graph Decomposition Method (GDM) to decompose the problem into several sub-problems, (3) TSP-based Heuristic (TSPH) to compute a circuit for UAV trajectory planning, (4) PoI-Station Mapping Method (PISMM) to match each PoI to a bus station, (5) Initial State Generation Method (ISGM) to generate initial states of UAVs and (6) Time Slot Scheduling Method (TSSM) to schedule UAVs for each time slot. These components will be detailed in the following subsections.

B. Complete Digraph Construction Method

By taking the bus stations as nodes and bus routes as the pathes among these nodes, the obtained graph is a directed connected graph. However, the graph may be not a complete digraph since not every pair of bus stations are directly connected. We need to build an artificial complete digraph by Complete Digraph Construction Method (CDCM). In Algorithm 2, there are two cases for the edges. In the first case (Lines 3-5) for two directly

Algorithm 1: Public Transportation-Assisted UAV Scheduling Framework.

```

1 Call Complete Digraph Construction Method to construct a complete
  digraph  $\mathbb{G}$  on all bus stations;
2 Perform Graph Decomposition Method on  $\mathbb{G}$  to obtain  $K$ 
  sub-graphs  $\mathbb{G}_1, \dots, \mathbb{G}_K$ ;
3 for  $\forall \mathbb{G}_\kappa \in \{\mathbb{G}_1, \dots, \mathbb{G}_K\}$  do
4   Perform TSP-based Heuristics on  $\mathbb{G}_\kappa$  to compute a circuit  $\mathbb{H}_\kappa$ ;
5   Call PoI-Station Mapping Method to generate matching set  $\mathbb{M}_\kappa$ 
     of PoIs and bus stations in  $\mathbb{G}_\kappa$ ;
6 Perform Initial State Generation Method on every pair of  $\mathbb{H}_\kappa$  and
   $\mathbb{M}_\kappa$  to determine the initial states of all UAVs:  $\pi_{1,1}, \dots, \pi_{n,1}$ ;
7  $W \leftarrow \emptyset$ ; /* Set of working UAVs */
8  $I \leftarrow \emptyset$ ; /* Set of idle UAVs */
9  $C \leftarrow \emptyset$ ; /* Set of charging UAVs */
10 for  $i = 1$  to  $n$  do
11   if  $p_{i,1} \in \mathbb{P}$  then
12      $W \leftarrow W + \{u_i\}$ ;
13   if  $p_{i,1} = P_0$  then
14      $I \leftarrow I + \{u_i\}$ ;
15   else
16      $C \leftarrow C + \{u_i\}$ ;
17  $T \leftarrow \varepsilon$ ;
18 while System is not terminated do
19   Update the states of UAVs at time  $T$ ;
20   Check the feasibilities of UAV states at time  $T$ ;
21   if Exist unfeasible UAV state then
22     break;
23   for  $\forall \mathbb{G}_\kappa \in \{\mathbb{G}_1, \dots, \mathbb{G}_K\}$  do
24     Perform Time Slot Scheduling Method
25     TSSM( $T, W, I, C, \mathbb{G}_\kappa$ ) to schedule all UAVs provisioned
       in  $\mathbb{G}_\kappa$ ;
26    $T \leftarrow T + \varepsilon$ ;
27  $T \leftarrow T - \varepsilon$ ;
28 return  $T$ ; /* The system life time is returned */

```

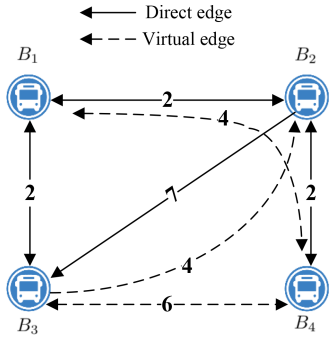


Fig. 5. Artificial Complete Digraph.

connected bus stations, the weight of edge between them is their distance. In the second case (Lines 7-9) for two bus stations having no direct edge, the shortest path between them is taken as their virtual edge. The weight of the edge is the length of the shortest path. Finally, the artificial completed digraph is returned (Line 10). As in the example shown in Fig. 5, B_3 and B_4 are not directly connected. The shortest path between them are $\langle B_3, B_1, B_2, B_4 \rangle$ and $\langle B_4, B_2, B_1, B_3 \rangle$, hence virtual edges $\langle B_3, B_4, 6 \rangle$ and $\langle B_4, B_3, 6 \rangle$ are added. The virtual edges $\langle B_1, B_4, 4 \rangle$, $\langle B_4, B_1, 4 \rangle$ and $\langle B_3, B_2, 4 \rangle$ are constructed in the similar manner. The shortest path algorithm could be Dijkstra algorithm or Floyd algorithm.

Algorithm 2: Complete Digraph Construction Method.

```

1  $\mathbb{V} \leftarrow \mathbb{V}$ ;
2  $\mathbb{E} \leftarrow \emptyset$ ;
3 for  $\forall B_j, B_{j'} \in \mathbb{V}$  do
4   if Exist a bus route where  $B_j$  is adjacent to  $B_{j'}$  then
5     Add edge  $\langle B_j, B_{j'}, D(B_j, B_{j'}) \rangle$  to  $\mathbb{E}$ ;
6  $\mathbb{G} \leftarrow \langle \mathbb{V}, \mathbb{E} \rangle$ ;
  /* Transform  $\mathbb{G}$  to a complete digraph by adding
  virtual edges */
7 for  $\forall \langle B_j, B_{j'} \rangle \notin \mathbb{E}$  do
8   Perform Shortest Path algorithm on  $\mathbb{G}$  to compute the shortest
    path from  $B_j$  to  $B_{j'}$ ;
  /* Suppose  $\lambda_{j,j'}$  is the length of the
  shortest path from  $B_j$  to  $B_{j'}$  */
9   Add a virtual edge  $\langle B_j, B_{j'}, \lambda_{j,j'} \rangle$  to  $\mathbb{E}$ ;
10 return  $\mathbb{G}$ ; /*  $\mathbb{G}$  is a complete digraph */

```

C. Graph Decomposition Method

Graph Decomposition Method is utilized to divide the considered problem into several independent sub-problems. These sub-problems will be conquered separately.

In Algorithm 3, K clusters of bus stations are obtained by the Clustering Method (Line 1). Corresponding K PoI sets are initialized empty (Line 2). Then all PoIs are inserted into these PoI sets (Line 3-7). If all bus stations which can serve P_h belong to the same cluster, P_h is assigned to the corresponding PoI set (Lines 4-5). Otherwise, one of related clusters (containing a bus station that can serve P_h) is selected by Roulette Wheel Selection and the corresponding PoI set will accommodate the PoI (Line 7). Roulette Wheel Selection is performed on the workload of clusters. The workload of a cluster can be described by the average number of PoIs per bus station in the cluster, i.e., $\frac{|\mathbb{P}_\kappa|}{|\mathbb{B}_\kappa|}$. A cluster with less workload would have more chance to be selected. Finally, K sub-graphs are constructed and returned (Lines 8-10). Since all related edges (either real or virtual edges) are included in a sub-graph, every sub-graph is a complete digraph.

Three GDM variants are proposed with different Clustering Methods.

- GDM1 does not perform any clustering method and returns the original graph, i.e., $K = 1$.
- GDM-KM employs K -Means clustering algorithm to generate K clusters.
- GDM-Kruskal uses Kruskal algorithm to generate K clusters. Kruskal algorithm is developed for minimum spanning tree. It can also be used as an implementation of K -clustering.

D. TSP-Based Heuristic

TSPH is performed on the complete digraph obtained by CDCM to constitute a tour that includes all bus stations on a given graph with the minimal travel distance. For each complete digraph obtained by CDCM, a TSP is modeled. Among various heuristics for TSP, two heuristics for solving TSP are employed: Lin-Kernighan heuristic (LKH) and Pointer Network (Ptr-Net).

- Lin-Kernighan heuristic (LKH) [27] is an effective implementation for solving the traveling salesman problem. LKH defines $5 - opt$ as a basic operation where five edges are removed from a tour and new five edges are added to constitute a new tour. LKH starts from a random feasible

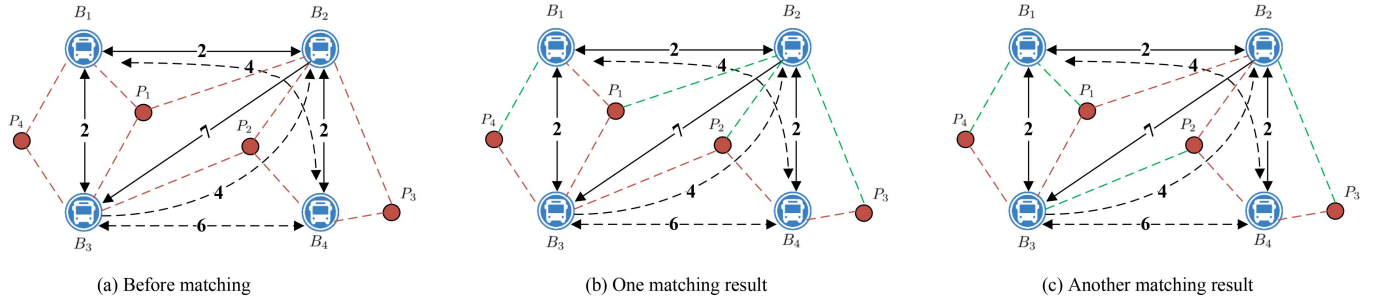


Fig. 6. Examples of PISMM.

Algorithm 3: Graph Decomposition Method.

```

1 Perform Clustering Method to divide all bus stations into  $K$  sets
 $\mathbb{B}_1, \dots, \mathbb{B}_K$ ;
2 Initialize  $K$  empty PoI sets  $\mathbb{P}_1, \dots, \mathbb{P}_K$ ;
3 for  $\forall P_h \in \mathbb{P}$  do
4   if  $\mathcal{B}(P_h) \in \mathcal{B}_\kappa$  and  $B_\kappa \in \{\mathbb{B}_1, \dots, \mathbb{B}_K\}$  then
5      $\mathbb{P}_\kappa \leftarrow \mathbb{P}_\kappa \cup \{P_h\}$ ;
6   else
7     Perform Roulette Wheel Selection to select a PoI set  $\mathbb{P}_\kappa$  to
       accommodate  $P_h$ ;
8 for  $\kappa = 1$  to  $K$  do
9   Construct sub-graph  $\mathbb{G}_\kappa$  which is composed of  $\mathbb{B}_\kappa, \mathbb{P}_\kappa$  and all
     related edges in original graph.
10 return  $\mathbb{G}_1, \dots, \mathbb{G}_K$ ; /*  $K$  sub-graphs are returned */

```

Algorithm 4: PoI-Station Mapping Method PISMM ($\mathbb{G}_\kappa$).

```

1 Randomly select two PoI  $P_{h_1}$  and  $P_{h_2}$  in  $\mathbb{G}_\kappa$ ;
2 Compute the closest pair of bus stations  $B_{j_1} \in \mathcal{B}(P_{h_1})$  and
 $B_{j_2} \in \mathcal{B}(P_{h_2})$  based on graph  $\mathbb{G}_\kappa$ ;
3  $\mathbb{M}_\kappa \leftarrow \{ \langle P_{h_1}, B_{j_1} \rangle, \langle P_{h_2}, B_{j_2} \rangle \}$ ;
4 for  $\forall P_h$  not matched do
5   Randomly select a matching pair  $\langle P_{h'}, B_{j'} \rangle \in \mathbb{M}_\kappa$ ;
6   Obtain the bus station  $B_j \in \mathcal{B}(P_h)$  which is closest to
 $B_{j'}$  on graph  $\mathbb{G}_\kappa$ ;
7   Generation a new matching pair  $\langle P_h, B_j \rangle$ ;
8    $\mathbb{M}_\kappa \leftarrow \mathbb{M}_\kappa \cup \{ \langle P_h, B_j \rangle \}$ ;
9 return  $\mathbb{M}_\kappa$ ; /* Matching set of PoIs and bus
stations */

```

tour and repeatedly performs the basic operations to improve the tour until no improvement can be made.

- Pointer Net (Ptr-Net) [28] is a neural architecture which can learn the conditional probability of an output sequence with elements that are discrete tokens corresponding to positions in an input sequence. It shows advantage for solving TSP.

For example, the best tour obtained by TSPH in Fig. 5 is $\langle B_1, B_2, B_4, B_3, B_1 \rangle$. However, there is no direct connection between B_4 and B_3 . We use the virtual edge $\langle B_4, B_3, 6 \rangle$ to compute the best tour. The real tour to visit all bus stations in one round is $\langle B_1, B_2, B_4, B_2, B_1, B_3, B_1 \rangle$ by replacing the virtual edge $\langle B_4, B_3, 6 \rangle$ with the real path $\langle B_4, B_2, B_1, B_3 \rangle$. A tour to visit all bus stations in $\mathbb{G}_\kappa$ is called a circuit $\mathbb{H}_\kappa$, which will be used as the UAV trajectory plan.

E. PoI-Station Mapping Method

In PoI-Station Mapping Method (PISMM), each PoI is originally assigned to one bus station. A match pair $\langle P_h, B_j \rangle$ indicates that one UAV should be originally provisioned at bus station B_j to serve for PoI P_h .

Algorithm 4 details the procedure of PISMM. First, two PoIs are randomly selected (Line 1). The closest pair of bus stations where two PoIs can be served are chosen (Line 2) and the first two matches are made accordingly (Line 3). Then the rest of PoIs are assigned to bus stations in an iterative manner (Lines 4-8). For each unmatched PoI, one matching pair are randomly selected (Line 5). The bus station closest to the selected matching pair is assigned to the current PoI (Line 6) and the matching pair is made accordingly (Lines 7-8). Finally, Matching set of PoIs and bus stations is returned (Line 9).

Fig. 6 shows examples of PISMM. In Fig. 6(a), a red dotted line connects one PoI and one bus station where the PoI can be served. We have $\mathcal{B}(P_1) = \{B_1, B_2, B_3\}$, $\mathcal{B}(P_2) = \{B_2, B_3, B_4\}$, $\mathcal{B}(P_3) = \{B_2, B_4\}$ and $\mathcal{B}(P_4) = \{B_1, B_3\}$. First, P_1 and P_2 are selected and the closest pair of bus stations serving for them is (B_2, B_2) . The corresponding two matches are $\langle P_1, B_2 \rangle$ and $\langle P_2, B_2 \rangle$. Suppose the match $\langle P_2, B_2 \rangle$ is selected to generate a new match for P_3 . The closest bus station in $\mathcal{B}(P_3)$ to B_2 is B_2 itself. Therefore, P_3 is assigned to B_2 . Finally, the match $\langle P_1, B_2 \rangle$ is selected to generate a new match for P_4 . The closest bus station in $\mathcal{B}(P_4)$ to B_2 is B_1 , so the match is $\langle P_4, B_1 \rangle$. The matching result is shown in Fig. 6(b) where the green dotted lines connect the matching pairs. If the PoIs are matched in different order, the matching result would be different. Fig. 6(c) shows another matching result. It is obtained by first generating matching pairs for P_3 and P_4 . Then $\langle P_4, B_1 \rangle$ is selected to generate the match for P_1 , i.e., $\langle P_1, B_1 \rangle$. Finally $\langle P_4, B_1 \rangle$ is again selected to generate the match for P_2 , i.e., $\langle P_2, B_3 \rangle$.

F. Initial State Generation Method

Initial State Generation Method (ISGM) generates initial states for all UAVs. Algorithm 5 describes the procedure of ISGM.

First, the UAVs of the working states are determined by the matching pairs in all clusters (Lines 2-6). For each matching pair $\langle P_h, B_j \rangle$, a UAV u_i is randomly selected and allocated to B_j (Line 3). u_i is under the working state serving for P_h at the bus station B_j . The state starts at time 0 and is assumed to last until u_i is out of power. The completion time of the state is computed based on that assumption (Line 4). The first state of

Algorithm 5: Initial State Generation Method ISGM ($\mathbb{H}, \mathbb{M}$).

```

1  $U \leftarrow \{u_1, \dots, u_n\}$ ;
2 for  $\forall \langle P_h, B_j \rangle \in \bigcup_{\kappa=1}^K \mathbb{M}_\kappa$  do
3   Randomly select  $u_i$  and set  $L_{i,0} = B_j$ ;
4    $c_{i,1} \leftarrow \frac{E_{max}}{e + e_m x_h + e_r y_h}$ ;
5    $\pi_{i,1} \leftarrow \langle P_h, B_j, B_j, 0, c_{i,1} \rangle$ ; /* Working state */
6    $U \leftarrow U - \{u_i\}$ ;
7 Distribute UAVs in  $U$  equably to all bus stations;
8 for  $\forall u_i \in U$  do
9    $B \leftarrow L_{i,0}$ ;
10  /* Suppose  $B$  belongs to  $\mathbb{G}_\kappa$  */
11  Obtain the next bus station  $B'$  of  $B$  on  $\mathbb{H}_\kappa$ ;
12  if  $\exists b_k \in BS(0, B)$  and  $r_{k,1} = B, r_{k,2} = B'$  then
13     $\pi_{i,1} \leftarrow \langle b_k, B, B', 0, T_{k,2} \rangle$ ; /* Charging state */
14  else
15     $c_{i,1} \leftarrow \frac{E_{max}}{e}$ ;
16     $\pi_{i,1} \leftarrow \langle P_0, B, B, 0, c_{i,1} \rangle$ ; /* Idle state */
17 return  $\pi_{1,1}, \pi_{2,1}, \dots, \pi_{n,1}$ ; /* Initial states of all UAVs are returned */

```

u_i is generated accordingly (Line 5). Then the rest of UAVs are distributed to bus stations equably (Line 7). Finally, the states of the rest UAVs are determined to be either idle or charging state (Lines 8-15). Given a UAV u_i to arrange, suppose its current location is B and the next bus station on the circuit $\mathbb{H}$ is B' (Lines 9-10). If there is a bus waiting at B at time 0 and its next stop on the route happens to be B' (Line 11), then u_i will take the bus and heads for B' . The state of u_i is set to charging state accordingly (Line 12). Otherwise, u_i is set to idle state (Lines 14-15). The state is assumed to last until u_i is out of power (Line 14). Finally, the initial states of all UAVs are returned (Line 16).

G. Time Slot Scheduling Method

Time Slot Scheduling Method (TSSM) schedules all UAVs by transiting states of UAVs. Algorithm 6 shows the procedure of state transition.

First of all, all idle UAVs in $\mathbb{G}_\kappa$ are checked (Lines 1-6). An idle UAV can be either transited to charging state or stayed idle. For each idle UAV u_i , its current state $\pi_{i,q}$ is obtained. Suppose its current location is B (Line 2) and the next bus station on the circuit $\mathbb{H}_\kappa$ is B' (Line 3). If there is a bus waiting at B at time T and its next stop on the route happens to be B' (Line 4), then u_i will take the bus and heads for B' . The current idle state is reset (Line 5) and u_i is transited to a new charging state (Line 6). Otherwise, u_i would keep idle.

Then all charging UAVs in $\mathbb{G}_\kappa$ are checked (Lines 7-30). A charging UAV can be either transited to working state or stayed charging. If a charging UAV arrives at its destination of the current charging state, i.e., the completion time of its current state is equal to current time T (Line 8), then the following steps are performed:

- 1) The current location of u_i is obtained (Line 9) and a set of candidate UAVs is generated by Candidate Selection Method (Line 10). Candidate UAVs are working UAVs. The serving object of one candidate UAV would be taken over by u_i .
- 2) If there is no candidate UAV (Line 11), then u_i will keep charging or become idle depending on whether it can take some bus to charge. u_i will head for B' which is the next

bus station of B on circuit $\mathbb{H}_\kappa$ (Line 12). If there is a bus which can take u_i to B' at time T (Line 13), u_i is transited to a new charging state (Line 14). Otherwise, it is transited to an idle state (Lines 16-17).

- 3) If there are candidate UAVs, the one with the minimal residual energy is selected (Line 19). u_i will take over the serving job from the candidate UAV $u_{i'}$ and it is transited to a new working state (Lines 20-22). The current working state of $u_{i'}$ is reset accordingly (Lines 23-24). Similarly, $u_{i'}$ is transited to either a charging state or an idle state depending on if it can take a bus to charge (Lines 25-30).

At the end of Algorithm 6, the set W of working UAVs, the set I of idle UAVs and the set C of charging UAVs are updated.

Apparently, a working UAV stays working if no UAV take over its job. If its job is taken over by other UAV, it would become charging state if an appropriate bus is available at that moment. Otherwise, it becomes idle.

The key component of TSSM is the Candidate Selection Method (CSM). Given the current bus station B and the set W of working UAVs, two CSMs are proposed.

- CSM1 selects all UAVs working at B as candidate UAVs, i.e., $\forall u_i, u_i \in W$ and $u_i \in US(T, B)$.
- CSM2 selects all UAVs working for PoIs which can be accessed from B , i.e., $\forall u_i, u_i \in W$ and $p_{i,q} \in \mathcal{P}(B)$ (suppose the current state of u_i is $\pi_{i,q}$).

V. EXPERIMENTS

In the section, we first calibrate the components of the proposed framework PTUSF. Then the framework with the best combination of the components is compared with several baseline algorithms for the similar problems. All compared algorithms are encoded in Java and run on an AMD Ryzen 7 8745H processor with Radeon 780M Graphics, 16 GB of RAM, running a 64-bit operating system and 512 GB of SSD.

A. Test Instances

The test instances are generated based on the study investigating the similar problem [10]. The main parameters settings for generating test instances are given in Table V. The meaning of these parameters can be seen in Table I.

Based on Table V, the main procedure to generate a test instance is given below.

- We prepare three maps Map_1, Map_2, Map_3 of different sizes to simulate the distribution of bus stations and PoIs. The size of these maps are 1000 m $\times$ 1000 m, 2000 m $\times$ 1500 m and 2500 m $\times$ 2000 m, respectively.
- The bus stations are distributed randomly in a given map. Then we randomly place PoIs in the area of 400 meters around bus stations. For a bus station B_j , $\mathcal{P}(B_j)$ consists of PoIs within 400 meters of B_j .
- For each PoI, its requirement is randomly set to $< 1, 1 >$, $< 1, 0 >$ and $< 0, 1 >$.
- The number of UAVs varies with the number of PoIs. We introduce the UAV quantity factor θ_{uav} to describe the linear relationship between the number of PoIs and the number of UAVs. n is set to $\theta_{uav} \times H$. The initial locations ($L_{i,0}$) of UAVs are set randomly at bus stations.
- The number of routes is denoted by N_r . Each route r_k is defined by a sequence of n_k bus stations, randomly selected from the set $\mathbb{B}$ such that $n_k \in [8, \frac{M}{2}]$. Routes can operate

Algorithm 6: Time Slot Scheduling Method TSSM
 $(T, W, I, C, \mathbb{G}_K)$.

```

1 for  $\forall u_i \in I$  do
  /* Suppose the current state of  $u_i$  is  $\pi_{i,q}$  */
2   $B \leftarrow l_{i,q}$ ; /*  $B$  is the current location of  $u_i$  */
  /*
3  Obtain the next bus station  $B'$  of  $B$  on  $\mathbb{H}_K$ ;
4  if  $\exists b_k \in BS(T, B)$  and  $\exists f, r_{k,f} = B, r_{k,f+1} = B'$  then
5     $\pi_{i,q} \leftarrow \langle P_0, B, B', s_{i,q}, T \rangle$ ; /* Reset idle state */
6     $\pi_{i,q+1} \leftarrow \langle b_k, B, B', T, T_{k,f+1} \rangle$ ; /* New charging state */
  /*
7 for  $\forall u_i \in C$  do
  /* Suppose the current state of  $u_i$  is  $\pi_{i,q}$ 
  and  $u_i$  arrives at the destination */
8  if  $c_{i,q} = T$  then
9     $B \leftarrow l'_{i,q}$ ;
10   Perform Candidate Selection Method CSM( $B, W$ ) to
   obtain a set of candidate working UAVs;
11   if No candidate UAV exists then
12     Obtain the next bus station  $B'$  of  $B$  on  $\mathbb{H}_K$ ;
13     if  $\exists b_k \in BS(T, B)$  and  $\exists f, r_{k,f} = B, r_{k,f+1} = B'$ 
       then
14        $\pi_{i,q+1} \leftarrow \langle b_k, B, B', T, T_{k,f+1} \rangle$ ; /* New
         charging state */
15     else
16        $c_{i,q+1} \leftarrow \frac{R(u_i, c_{i,q})}{e}$ ;
17        $\pi_{i,q+1} \leftarrow \langle P_h, B, B, T, c_{i,q+1} \rangle$ ; /* New
         idle state */
18   else
19     Select candidate  $u_{i'}$  with the minimal residual energy;
     /* Suppose the current working state
     of  $u_{i'}$  is  $\pi_{i',q'}$  */
20      $P_h \leftarrow p_{i',q'}$ ;
21      $c_{i,q+1} \leftarrow \frac{R(u_i, c_{i,q})}{e + e_m x_h + e_r y_h}$ ;
22      $\pi_{i,q+1} \leftarrow \langle P_h, B, B, T, c_{i,q+1} \rangle$ ; /* New
       working state for  $u_i$  */
23      $B' \leftarrow l_{i',q'}$ ; /*  $B'$  is the current
       location of  $u_{i'}$  */
24      $\pi_{i',q'} \leftarrow \langle P_h, B', B', s_{i',q'}, T \rangle$ ; /* Reset
       working state of  $u_{i'}$  */
25     Obtain the next bus station  $B''$  of  $B'$  on  $\mathbb{H}_K$ ;
26     if  $\exists b_k \in BS(T, B')$  and
        $\exists f, r_{k,f} = B', r_{k,f+1} = B''$  then
27       /* New charging state */
        $\pi_{i',q'+1} \leftarrow \langle b_k, B', B'', T, T_{k,f+1} \rangle$ ;
28     else
29        $c_{i',q'+1} \leftarrow \frac{R(u_{i'}, c_{i',q'})}{e}$ ;
       /* New idle state */
30        $\pi_{i',q'+1} \leftarrow \langle P_0, B', B', T, c_{i',q'+1} \rangle$ ;
31 Update the set  $W$  of working UAVs, the set  $I$  of idle UAVs and the
   set  $C$  of charging UAVs, accordingly;

```

as either round-trip or circular services. Bus frequency is fixed across all routes, with one departure every 15 minutes per route. Therefore the number of required buses m is not a fixed number. It depends on the system lifetime.

- DJI Phantom 4 Pro V2.0 is used as the UAV model. The maximum battery capacity of UAVs $E_{\max}$ is set to 3.2112×10^5 joules.

The parameter settings in Table V are chosen to reflect realistic urban surveillance scenarios and align with values commonly used in related literature. The number of bus stations (M) and points of interest (H) are varied to evaluate scalability. The UAV

TABLE V
PARAMETER SETTINGS FOR TEST INSTANCES.

Notation	Value	Notation	Value
M	20,30,40	N_r	5,10
v	5 m/s	H	5, 10, ..., 20
n_k	$[8, \frac{M}{2}]$	ϵ	1 s
$w\epsilon$	5 s	$E_{\max}$	3.2112×10^5 joules
θ_{uav}	1.5, 2, 2.5, 3	n	$\theta_{uav} \times H$
e	100 watts	e_m	20 watts
e_r	50 watts	e_c	25 watts

quantity factor (θ_{uav}) is introduced to maintain a proportional relationship between UAVs and PoIs, ensuring sufficient coverage without excessive resource allocation. Energy parameters (e, e_m, e_r, e_c) are based on typical power consumption profiles of the commercial UAV model DJI Phantom 4 Pro. We employ this UAV model because it is a popular choice in academic research for UAVs. The bus speed (v), dwell time ($w\epsilon$), and route structure (N_r, n_k) are set to mimic real-world public transportation networks. These values collectively ensure that the test instances are practically relevant.

There are $3 \times 5 \times 3 \times 4 \times 2 = 360$ instance combinations (three maps, $H \in \{5, 10, 15, 20, 25\}$, $M \in \{20, 30, 40\}$, $\theta_{uav} \in \{1.5, 2, 2.5, 3\}$, $N_r \in \{5, 10\}$).

B. Component Calibration

In the proposed framework PTUSF, there are three variants for GDM (GDM1, GDM-KM and GDM-Kruskal), two variants for TSPH (LKH and Ptr-Net) and two variants for CSM (CSM1 and CSM2). In total, there are $3 \times 2 \times 2 = 12$ component variants for PTUSF. For each instance combination, 10 test instances are randomly generated. The component variants are calibrated on $360 \times 5 = 1,800$ test instances in total. Therefore, $12 \times 1,800 = 21,960$ experimental results are obtained.

The experimental results obtained by these variants are measured by the relative percentage deviation (RPD) as defined by (23). Where in $T_{\max}$ is the maximum system lifetime obtained by all the compared algorithms. All the experimental results are analyzed by the multi-factor analysis of variance (ANOVA) technique.

$$RPD(\mathbb{S}) = \frac{T_{\max} - T(\mathbb{S})}{T_{\max}} \times 100\% \quad (23)$$

Fig. 7(a) shows the RPD of variants with different CSM components with 95.0% Tukey Honest Significant Difference (HSD) intervals. It can be observed that the variant with CSM2 outperforms the one with CSM1. The reason lies in that a UAV may have more candidates to take over in CSM2 than that in CSM1. The more frequent the takeover action takes place among UAVs, the longer the system life time would be.

Fig. 7(b) shows the RPD of variants with different GDM components with 95.0% Tukey Honest Significant Difference (HSD) intervals. For GDM-KM and GDM-Kruskal, the cluster number K is set to 5. It shows a statistically significant difference for the performance of different GDM components. GDM-KM has the best performances among all the GDM components.

Fig. 7(c) shows the RPD of variants with different TSPH components with 95.0% Tukey Honest Significant Difference (HSD) intervals. Fig. 7(d) compares the variants on computation time. It is shown that there is no significant difference on the RPD

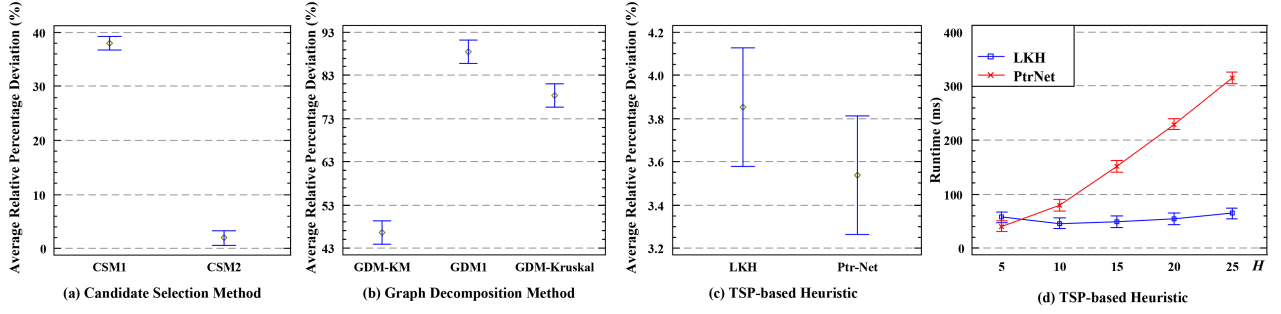


Fig. 7. Mean plots of variants with different components across all instances with 95.0% Tukey HSD intervals.

TABLE VI
EXPERIMENTAL RESULTS.

Parameter	Value	ARPD (%)				RI (%)			Time (ms)			
		GH	MACO	DRL	PTUSF	GH	MACO	DRL	GH	MACO	DRL	PTUSF
H	5	8.51	4.95	5.99	5.70	3.70	-0.19	0.96	0.010	15.57	50.36	791.54
	10	8.57	6.12	6.29	5.51	3.98	1.29	1.50	0.006	51.84	74.64	305.06
	15	7.59	6.39	6.03	4.58	3.87	2.58	2.18	0.008	53.78	97.51	113.70
	20	6.67	5.99	5.53	3.92	3.46	2.82	2.30	0.043	54.49	119.52	119.24
	25	5.99	5.37	5.26	4.10	2.43	1.84	1.76	0.004	56.53	148.34	123.58
θ_{uav}	1.5	6.30	5.02	4.72	3.70	3.19	1.81	1.56	0.037	46.60	156.89	474.70
	2	7.78	5.72	6.05	4.78	3.79	1.64	1.95	0.006	47.33	79.28	231.81
	2.5	8.04	6.38	5.99	4.98	3.93	2.19	1.74	0.006	46.98	80.30	115.47
	3	7.57	6.04	6.56	5.53	2.92	1.20	1.81	0.006	47.49	78.60	267.51
M	20	6.33	5.06	4.78	4.41	2.46	1.16	0.81	0.030	47.06	136.78	336.75
	30	7.94	6.11	6.47	4.63	4.26	2.28	2.68	0.006	47.76	80.36	308.11
	40	8.05	6.22	6.27	5.15	3.78	1.79	1.91	0.006	46.41	79.17	177.39
N_f	5	7.35	5.52	5.44	4.59	3.51	1.58	1.50	0.023	46.74	120.24	340.70
	10	7.44	6.02	6.16	4.84	3.40	1.86	2.03	0.006	47.44	80.00	212.55
Map	Map1	7.68	3.89	4.06	3.38	5.18	1.03	1.19	0.029	48.09	100.23	470.50
	Map2	6.59	6.05	6.38	6.24	0.92	0.33	0.69	0.007	47.98	98.64	178.57
	Map3	7.89	7.47	7.04	4.59	4.17	3.83	3.43	0.007	45.13	102.39	172.55
Average		7.39	5.77	5.79	4.71	3.46	1.72	1.76	0.015	47.08	100.42	277.57

metric for LKH and Ptr-Net. However, the computation time of Ptr-Net increases with the number of PoIs. LKH outperforms Ptr-Net on efficiency.

Therefore, the best component combination of PTUSF is GDM-KM for GDM, LKH for TSPH and CSM2 for CSM.

C. Algorithm Comparison

The proposed PTUSF is compared with three baseline algorithms for the UAV trajectory planning problems: Greedy Heuristics (GH) [10], modified Ant Colony Optimization (MACO) algorithm [23] and the Deep Reinforcement Learning-based (DRL) algorithm [21]. GH generates the UAV trajectory plan using a greedy strategy. MACO constructs efficient paths for UAVs by an adapted version of the ACO algorithm. DRL introduces a novel deep reinforcement learning technique to efficiently learn from UAV trajectory policy experiences. While the study in [21] does not involve bus stations, its DRL framework provides a valuable foundation for our work. We adapt their approach to our problem through the following modifications: 1) incorporating bus stations, bus timetables and PoI locations as key environmental factors; 2) defining the state based on visited PoIs and current UAV locations; 3) redefining actions to include bus boarding decisions;

and 4) adjusting the reward function considering the route distance.

In order to demonstrate the improvement of the proposed PTUSF over compared methods, the Relative Improvement (RI) metric is defined based on the system lifetime obtained by PTUSF (24). RIM indicates the improvement that PTUSF extends system lifetime.

$$RI(\mathcal{S}) = \frac{T(\mathcal{S}_{PTUSF}) - T(\mathcal{S})}{T(\mathcal{S})} \times 100\% \quad (24)$$

To compare the algorithms, 10 new instances are randomly generated for each instance combination, i.e., $360 \times 10 = 3,600$ new test instances for comparison. The average RPD (ARPD) and computation time of the compared algorithms are given in Table VI.

Table VI shows that PTUSF achieves the best performance in the ARPD metric (average 4.71%) in most cases, followed by DRL (average 5.79%) and MACO (average 5.77%). Although DRL and MACO perform similarly and occasionally outperform PTUSF (e.g., when $H = 5$ or on Map2), their overall ARPDS remain inferior. In contrast, GH exhibits the worst performance across all instances (average 7.39%).

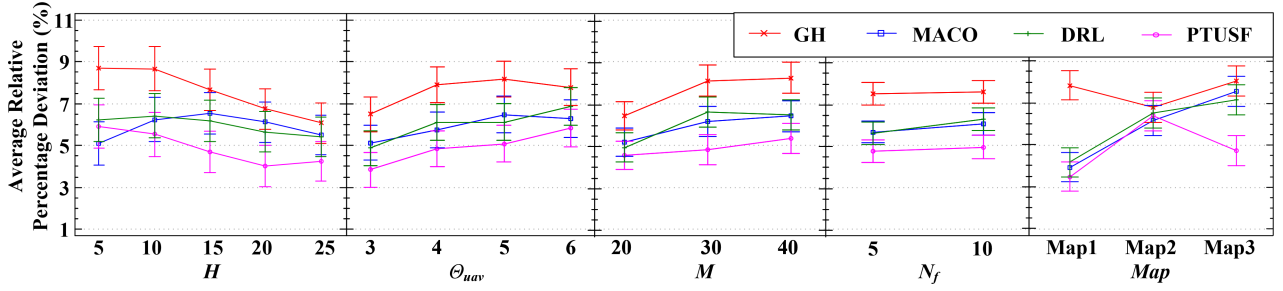


Fig. 8. Interactions between instance parameters and the compared algorithms with 95.0% Tukey HSD intervals.

According to the RI metric in Table VI, PTUSF achieves a 3.46% average improvement compared to GH in system lifetime, with gains reaching up to 5.18% in some cases ($Map = Map1$). PTUSF achieves an average 1.72% and a highest 3.83% ($Map = Map3$) improvement on system lifetime versus MACO. Against DRL approach, PTUSF shows a 1.7% average improvement and a 3.43% highest improvement ($Map = Map3$).

In terms of efficiency, GH is the fastest, with computation times under 1 ms, while MACO ranks second (around 50 ms), and DRL comes third (approximately 100 ms). PTUSF is the slowest, averaging 277.57 ms. However, all the algorithms compared are effective enough, since they take no more than 1 s to run on every instance. Fig. 8 depicts the interactions between instance parameters and the compared algorithms. Notably, PTUSF demonstrates strong robustness, showing minimal sensitivity to parameter variations, except for map changes. We can find that the performance of PTUSF varies significantly on different maps. According to the performance of compared algorithms on effectiveness, efficiency and robustness, we can conclude that PTUSF is suitable for solving the problem under study.

VI. CONCLUSION

In this paper, we investigated a multi-UAV scheduling problem in the public transportation-assisted urban surveillance system. The problem involves the energy constraint, full coverage constraint and consistency constraint, and the objective is to maximize the system lifetime. To solve the problem, we proposed the public transportation-assisted UAV scheduling framework (PTUSF) to obtain a promising solution. In the PTUSF, an artificial complete digraph was constructed and decomposed into several sub-graphs. The problem was divided into several independent sub-problems accordingly. For each sub-problem, an effective TSP heuristic was utilized for UAV trajectory planning. A time slot scheduling method was introduced for the task offloading problem, where all UAVs are scheduled by transiting states of UAVs time slot by time slot. Algorithm calibration was conducted to determine the best component for the PTUSF. By comparing with baseline algorithms, we have illustrated that the PTUSF outperforms the compared algorithms statistically on effectiveness and robustness. Although it is not the fastest one, it is effective enough for the problem under study.

REFERENCES

- [1] B. N. Silva, M. Khan, and K. Han, "Towards sustainable smart cities: A review of trends, architectures, components, and open challenges in smart cities," *Sustain. Cities Soc.*, vol. 38, pp. 697–713, 2018.
- [2] M. Shorfuazzaman, M. S. Hossain, and M. F. Alhamid, "Towards the sustainable development of smart cities through mass video surveillance: A response to the COVID-19 pandemic," *Sustain. Cities Soc.*, vol. 64, 2021, Art. no. 102582.
- [3] S. Shen, K. Yang, K. Wang, G. Zhang, and H. Mei, "Number and operation time minimization for multi-UAV-enabled data collection system with time windows," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 10149–10161, Jun. 2022.
- [4] I. Murturi and S. Dustdar, "A decentralized approach for resource discovery using metadata replication in edge networks," *IEEE Trans. Serv. Comput.*, vol. 15, no. 5, pp. 2526–2537, Sep./Oct. 2022.
- [5] B. Alkous, A. Abusafia, A. Lakhdari, and A. Bouguettaya, "In-flight energy-driven composition of drone swarm services," *IEEE Trans. Serv. Comput.*, vol. 16, no. 3, pp. 1919–1933, May/Jun. 2023.
- [6] B. Chen, H. Zhou, J. Yao, and H. Guan, "RESERVE: An energy-efficient edge cloud architecture for intelligent Multi-UAV," *IEEE Trans. Serv. Comput.*, vol. 15, no. 2, pp. 819–832, Mar./Apr. 2022.
- [7] Y. Jin, J. Xu, S. Wu, L. Xu, D. Yang, and K. Xia, "Bus network assisted drone scheduling for sustainable charging of wireless rechargeable sensor network," *J. Syst. Archit.*, vol. 116, 2021, Art. no. 102059.
- [8] Y. Jin, J. Xu, S. Wu, L. Xu, and D. Yang, "Enabling the wireless charging via bus network: Route scheduling for electric vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 3, pp. 1827–1839, Mar. 2021.
- [9] Y. Jin, Z. Qian, S. Gong, and W. Yang, "Learning transferable driven and drone assisted sustainable and robust regional disease surveillance for smart healthcare," *IEEE/ACM Trans. Comput. Biol. Bioinf.*, vol. 18, no. 1, pp. 114–125, Jan./Feb. 2021.
- [10] A. Trotta, F. Andreagiovanni, M. Di Felice, E. Natalizio, and K. R. Chowdhury, "When UAVs ride a bus: Towards energy-efficient city-scale video surveillance," in *Proc. IEEE Conf. Comput. Commun.*, 2018, pp. 1043–1051.
- [11] Y. Sun et al., "UAV and IoT-based systems for the monitoring of industrial facilities using digital twins: Methodology, reliability models, and application," *Sensors*, vol. 22, no. 17, 2022, Art. no. 6444.
- [12] I. Mademlis et al., "High-level multiple-UAV cinematography tools for covering outdoor events," *IEEE Trans. Broadcast.*, vol. 65, no. 3, pp. 627–635, Sep. 2019.
- [13] Z. Chen, Z. Hu, Z. Bao, and W. Xu, "UAV charging station planning and route optimization considering stochastic delivery demand," in *IEEE Trans. Transport. Electric.*, vol. 10, no. 4, pp. 9328–9341, Dec. 2024.
- [14] V. Hassija, V. Chamola, D. N. G. Krishna, and M. Guizani, "A distributed framework for energy trading between UAVs and charging stations for critical applications," *IEEE Trans. Veh. Technol.*, vol. 69, no. 5, pp. 5391–5402, May 2020.
- [15] H. Ma, W. Liu, G. Yang, H. Sun, D. Qu, and H. Yang, "Efficient charging coverage strategy of UAV based on improved deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 74, no. 8, pp. 11832–11844, Aug. 2025.
- [16] H. Huang, A. V. Savkin, and W. Ni, "Online UAV trajectory planning for covert video surveillance of mobile targets," *IEEE Trans. Automat. Sci. Eng.*, vol. 19, no. 2, pp. 735–746, Apr. 2022.
- [17] E. S. Rigas, P. Kolios, M. Mavrouniotis, and G. Ellinas, "Scheduling a fleet of drones for monitoring missions with spatial, temporal, and energy constraints," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 9, pp. 15133–15145, Sep. 2022.
- [18] Y. Wang, X. Wang, H. Huang, and H. Dai, "Practical optimizing UAV trajectory in wireless charging networks: An approximated approach," *IEEE Trans. Mobile Comput.*, vol. 24, no. 11, pp. 12550–12566, Nov. 2025, doi 10.1109/TMC.2025.3586457.

- [19] R. G. Ribeiro, L. P. Cota, T. A. M. Euzébio, J. A. Ramírez, and F. G. Guimarães, "Unmanned-aerial-vehicle routing problem with mobile charging stations for assisting search and rescue missions in postdisaster scenarios," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 52, no. 11, pp. 6682–6696, Nov. 2022.
- [20] B. Li, W. Liu, W. Xie, and X. Li, "Energy-efficient task offloading and trajectory planning in UAV-enabled mobile edge computing networks," *Comput. Netw.*, vol. 234, no. 10, pp. 1–8, 2023.
- [21] B. Zhu, E. Bedeer, H. H. Nguyen, R. Barton, and J. Henry, "UAV trajectory planning in wireless sensor networks for energy consumption minimization by deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 70, no. 9, pp. 9540–9554, Sep. 2021.
- [22] K. Wang, X. Zhang, L. Duan, and J. Tie, "Multi-UAV cooperative trajectory for servicing dynamic demands and charging battery," *IEEE Trans. Mobile Comput.*, vol. 22, no. 3, pp. 1599–1614, Mar. 2023.
- [23] A. Jaiswal, S. Shivateja, A. Hazra, N. Mazumdar, J. Singh, and V. G. Menon, "UAV-Enabled mobile RAN and RF-Energy transfer protocol for enabling sustainable IoT in energy-constrained networks," *IEEE Trans. Green Commun. Netw.*, vol. 8, no. 3, pp. 1118–1127, Sep. 2024.
- [24] H. Huang and A. V. Savkin, "Aerial surveillance in cities: When UAVs take public transportation vehicles," *IEEE Trans. Automat. Sci. Eng.*, vol. 20, no. 2, pp. 1069–1080, Apr. 2023.
- [25] J. Park, S. Choi, T. Kim, C. Lee, and S. Lee, "Public bus-assisted task offloading for UAVs," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 12, pp. 20561–20573, Dec. 2024.
- [26] H. Huang, A. V. Savkin, and C. Huang, "Reliable path planning for drone delivery using a stochastic time-dependent public transportation network," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 8, pp. 4941–4950, Aug. 2021.
- [27] K. Helsgaun, "An effective implementation of the Lin–Kernighan traveling salesman heuristic," *Eur. J. Oper. Res.*, vol. 126, no. 1, pp. 106–130, 2000.
- [28] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2015, vol. 28, pp. 2692–2700.



Jie Zhu received the PhD degree in applied computer science from the School of Computer Science and Engineering, Southeast University, Nanjing, in 2011. From November 2008 to November 2009, she was with the Department of Electrical and Computer Engineering, University of Western Ontario, London, ON, Canada, as a visiting student. She joined the Nanjing University of Post and Telecommunication, in 2014, where she is currently an associate professor with the School of Computer Science. Her research interests include machine scheduling, project

scheduling, workflow optimization, and cloud computing, among which task scheduling and resource provisioning in clouds are her current core research areas.



Pengfei He received the BEng degree from the Nanjing University of Posts and Telecommunications, Nanjing, China, in 2021. He is currently working toward the MS degree with the School of Computer Science, Nanjing University of Posts and Telecommunications. His primary research interests include path planning and task scheduling of UAVs.



Ying Liu received the BEng degree from the Nanjing University of Posts and Telecommunications, Nanjing, China, in 2024. She is currently working toward the MS degree with the School of Computer Science, Nanjing University of Posts and Telecommunications. Her primary research interests include path planning and task scheduling of UAVs.



Haiping Huang (Member, IEEE) received the BEng and MEng degrees in computer science and technology from the Nanjing University of Posts & Telecommunications, Nanjing, China, in 2002 and 2005, respectively, and the PhD degree in computer application technology from Soochow University, China, in 2009. From May 2013 to November 2013, he was a visiting scholar with the School of Electronics and Computer Science, University of Southampton, Southampton, U.K. He is currently a professor with the School of Computer Science and Technology, Nanjing University of Posts & Telecommunications. His research interests include information security and privacy protection of wireless sensor networks.



Fu Xiao (Senior Member, IEEE) received the PhD degree in computer science and technology from the Nanjing University of Science and Technology, Nanjing, China, in 2007. He is currently a professor and a PhD supervisor with the School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing. His research interests include wireless sensor networks.



Reza Malekian (Senior Member, IEEE) is a professor, the dean with the College of Science and Engineering, and the director with the Engineering Research Center at the American University of Armenia. He is also a full professor with the Department of Computer Science and Media Technology, Malmö University, Sweden. His research focuses on the IoT, sensor systems and networks. He is a chartered engineer (CEng) and a fellow of the British Computer Society. Apart from serving on the IEEE Technical Communities, including chair of the Steering Committee of the IEEE ComSoc Multimedia Communications Technical Committee, and initiative on the IEEE Academy on IoT, he has been chair of the IEEE IoT Technical Community's Conference and Events Committee. He is an area editor for the *IEEE Internet of Things Journal* and an associate editor for the *IEEE Sensors Journal*.